```
"""
Ouroboros - all-in-one Discord bot
Moderation + modlogs, giveaways, suggestions, utility, welcomer,
anti-raid, anti-nuke and server logs.

Every command works both as a prefix command (default ",") and as a slash command
"""
import asyncio
import logging
import os
import random
import re
import sqlite3
import time
from collections import defaultdict, deque
from datetime import timedelta
from typing import Literal, Optional

import discord
from discord.ext import commands, tasks
from deep_translator import GoogleTranslator

# ---------------------------------------------------------------- config
TOKEN = os.environ["DISCORD_TOKEN"]
PREFIX = os.getenv("PREFIX", ",")
DB_PATH = os.getenv("DB_PATH", "ouroboros.db")  # point at a persistent volume in
GOLD = 0xC08040

RAID_JOINS, RAID_WINDOW, RAID_LOCK = 5, 10, 60       # 5 joins in 10s => raid mod
MENTION_LIMIT = 8                                    # mass-mention spam threshol
NUKE_WINDOW = 10                                     # seconds
NUKE_LIMITS = {"channel_delete": 3, "role_delete": 3, "ban": 3, "kick": 3,
               "channel_create": 6, "role_create": 6}
BOT_PREFIXES = ("!", "?", ".", ",", "$", "%", "&", "+", "=", "~", ";", ">")

logging.basicConfig(level=logging.INFO)
log = logging.getLogger("ouroboros")

# ---------------------------------------------------------------- db
db = sqlite3.connect(DB_PATH)
db.row_factory = sqlite3.Row
db.executescript("""
CREATE TABLE IF NOT EXISTS settings(guild_id INTEGER, key TEXT, value TEXT, PRIMA
```

```
CREATE TABLE IF NOT EXISTS warns(id INTEGER PRIMARY KEY AUTOINCREMENT, guild_id I
    mod_id INTEGER, reason TEXT, ts INTEGER);
CREATE TABLE IF NOT EXISTS tags(guild_id INTEGER, name TEXT, content TEXT, author
CREATE TABLE IF NOT EXISTS giveaways(message_id INTEGER PRIMARY KEY, guild_id INT
    prize TEXT, winners INTEGER, end_ts INTEGER, ended INTEGER DEFAULT 0, host_id
CREATE TABLE IF NOT EXISTS suggestions(id INTEGER PRIMARY KEY AUTOINCREMENT, guil
    channel_id INTEGER, user_id INTEGER, content TEXT);
CREATE TABLE IF NOT EXISTS whitelist(guild_id INTEGER, user_id INTEGER, PRIMARY K
""")


def q(sql, *args):
    cur = db.execute(sql, args)
    db.commit()
    return cur


def get(gid, key, default=None):
    r = db.execute("SELECT value FROM settings WHERE guild_id=? AND key=?", (gid,
    return r["value"] if r else default


def put(gid, key, val):
    q("INSERT OR REPLACE INTO settings VALUES(?,?,?)", gid, key, str(val))


# ---------------------------------------------------------------- bot
intents = discord.Intents.default()
intents.members = True
intents.message_content = True


class Ouroboros(commands.Bot):
    async def setup_hook(self):
        giveaway_loop.start()
        await self.tree.sync()


bot = Ouroboros(
    command_prefix=commands.when_mentioned_or(PREFIX),
    intents=intents,
    activity=discord.Activity(type=discord.ActivityType.watching, name=f"the cycl
)

# ------------------------------------------------------------- helpers
UNITS = {"s": 1, "m": 60, "h": 3600, "d": 86400, "w": 604800}
```

```
def parse_dur(s: str):
    m = re.fullmatch(r"(\d+)\s*([smhdw])", s.strip().lower())
    return int(m[1]) * UNITS[m[2]] if m else None


def embed(title, desc=None, color=GOLD):
    return discord.Embed(title=title, description=desc, color=color, timestamp=di


async def send_log(guild, kind, emb):
    cid = get(guild.id, kind)
    ch = guild.get_channel(int(cid)) if cid else None
    if ch:
        try:
            await ch.send(embed=emb)
        except discord.HTTPException:
            pass


async def log_action(guild, action, target, mod, reason, color=0xE67E22):
    e = embed(action, color=color)
    e.add_field(name="User", value=f"{target} (`{target.id}`)")
    e.add_field(name="Moderator", value=str(mod))
    e.add_field(name="Reason", value=str(reason)[:1000], inline=False)
    await send_log(guild, "modlog", e)


async def dm(user, text):
    try:
        await user.send(text)
    except (discord.HTTPException, AttributeError):
        pass


def can_act(ctx, member: discord.Member):
    if member.id == ctx.author.id or member.id == ctx.guild.owner_id:
        return False
    if ctx.author.id != ctx.guild.owner_id and member.top_role >= ctx.author.top_
        return False
    return member.top_role < ctx.guild.me.top_role


# ---------------------------------------------------------- moderation
@bot.hybrid_command(description="Ban a user (member or by ID)")
```

```
@commands.guild_only()
@commands.has_permissions(ban_members=True)
@commands.bot_has_permissions(ban_members=True)
async def ban(ctx, user: discord.User, *, reason: str = "No reason given"):
    m = ctx.guild.get_member(user.id)
    if m and not can_act(ctx, m):
        return await ctx.send("I can't ban that member (role hierarchy).")
    await dm(user, f"You were banned from **{ctx.guild.name}**: {reason}")
    await ctx.guild.ban(user, reason=f"{ctx.author}: {reason}")
    await log_action(ctx.guild, "Ban", user, ctx.author, reason, 0xCC3333)
    await ctx.send(f"🔨 Banned **{user}**.")


@bot.hybrid_command(description="Unban a user by ID")
@commands.guild_only()
@commands.has_permissions(ban_members=True)
@commands.bot_has_permissions(ban_members=True)
async def unban(ctx, user: discord.User, *, reason: str = "No reason given"):
    try:
        await ctx.guild.unban(user, reason=f"{ctx.author}: {reason}")
    except discord.NotFound:
        return await ctx.send("That user isn't banned.")
    await log_action(ctx.guild, "Unban", user, ctx.author, reason, 0x33AA55)
    await ctx.send(f"✅ Unbanned **{user}**.")


@bot.hybrid_command(description="Ban then unban to delete a day of messages")
@commands.guild_only()
@commands.has_permissions(ban_members=True)
@commands.bot_has_permissions(ban_members=True)
async def softban(ctx, member: discord.Member, *, reason: str = "No reason given"
    if not can_act(ctx, member):
        return await ctx.send("I can't softban that member (role hierarchy).")
    await ctx.guild.ban(member, reason=f"Softban by {ctx.author}: {reason}", dele
    await ctx.guild.unban(member, reason="Softban")
    await log_action(ctx.guild, "Softban", member, ctx.author, reason)
    await ctx.send(f"🧹 Softbanned **{member}**.")


@bot.hybrid_command(description="Kick a member")
@commands.guild_only()
@commands.has_permissions(kick_members=True)
@commands.bot_has_permissions(kick_members=True)
async def kick(ctx, member: discord.Member, *, reason: str = "No reason given"):
    if not can_act(ctx, member):
        return await ctx.send("I can't kick that member (role hierarchy).")
```

```
    await dm(member, f"You were kicked from **{ctx.guild.name}**: {reason}")
    await member.kick(reason=f"{ctx.author}: {reason}")
    await log_action(ctx.guild, "Kick", member, ctx.author, reason)
    await ctx.send(f"👢 Kicked **{member}**.")


@bot.hybrid_command(description="Timeout a member, e.g. /mute @user 10m")
@commands.guild_only()
@commands.has_permissions(moderate_members=True)
@commands.bot_has_permissions(moderate_members=True)
async def mute(ctx, member: discord.Member, duration: str, *, reason: str = "No r
    secs = parse_dur(duration)
    if not secs:
        return await ctx.send("Duration like `10m`, `2h`, `1d` (s/m/h/d/w).")
    if not can_act(ctx, member):
        return await ctx.send("I can't mute that member (role hierarchy).")
    secs = min(secs, 28 * 86400)
    await member.timeout(timedelta(seconds=secs), reason=f"{ctx.author}: {reason}
    await log_action(ctx.guild, f"Mute ({duration})", member, ctx.author, reason)
    await ctx.send(f"🔇 Muted **{member}** for {duration}.")


@bot.hybrid_command(description="Remove a timeout")
@commands.guild_only()
@commands.has_permissions(moderate_members=True)
@commands.bot_has_permissions(moderate_members=True)
async def unmute(ctx, member: discord.Member, *, reason: str = "No reason given")
    await member.timeout(None, reason=f"{ctx.author}: {reason}")
    await log_action(ctx.guild, "Unmute", member, ctx.author, reason, 0x33AA55)
    await ctx.send(f"🔊 Unmuted **{member}**.")


@bot.hybrid_command(description="Warn a member (auto-punishes at thresholds)")
@commands.guild_only()
@commands.has_permissions(moderate_members=True)
async def warn(ctx, member: discord.Member, *, reason: str = "No reason given"):
    if not can_act(ctx, member):
        return await ctx.send("I can't warn that member (role hierarchy).")
    q("INSERT INTO warns(guild_id,user_id,mod_id,reason,ts) VALUES(?,?,?,?,?)",
      ctx.guild.id, member.id, ctx.author.id, reason, int(time.time()))
    n = db.execute("SELECT COUNT(*) c FROM warns WHERE guild_id=? AND user_id=?",
                   (ctx.guild.id, member.id)).fetchone()["c"]
    await dm(member, f"You were warned in **{ctx.guild.name}**: {reason}")
    await log_action(ctx.guild, f"Warn (#{n})", member, ctx.author, reason, 0xF1C
    msg = f"⚠️ Warned **{member}** (warning #{n})."
    mute_at = int(get(ctx.guild.id, "warn_mute", 3))
```

```
    ban_at = int(get(ctx.guild.id, "warn_ban", 5))
    try:
        if ban_at and n >= ban_at:
            await ctx.guild.ban(member, reason=f"Reached {n} warnings")
            await log_action(ctx.guild, "Auto-ban (warnings)", member, bot.user,
            msg += " Auto-banned."
        elif mute_at and n >= mute_at:
            await member.timeout(timedelta(hours=1), reason=f"Reached {n} warning
            await log_action(ctx.guild, "Auto-mute 1h (warnings)", member, bot.us
            msg += " Auto-muted for 1h."
    except discord.HTTPException:
        msg += " (Auto-punishment failed: check my permissions.)"
    await ctx.send(msg)


@bot.hybrid_command(description="Show a member's warnings")
@commands.guild_only()
@commands.has_permissions(moderate_members=True)
async def warns(ctx, member: discord.Member):
    rows = db.execute("SELECT * FROM warns WHERE guild_id=? AND user_id=? ORDER B
                      (ctx.guild.id, member.id)).fetchall()
    if not rows:
        return await ctx.send(f"{member} has no warnings.")
    lines = [f"`#{r['id']}` <t:{r['ts']}:d> by <@{r['mod_id']}>: {r['reason'][:10
    await ctx.send(embed=embed(f"Warnings for {member} ({len(rows)})", "\n".join(


@bot.hybrid_command(description="Clear all warnings for a member")
@commands.guild_only()
@commands.has_permissions(moderate_members=True)
async def clearwarns(ctx, member: discord.Member):
    q("DELETE FROM warns WHERE guild_id=? AND user_id=?", ctx.guild.id, member.id
    await log_action(ctx.guild, "Warnings cleared", member, ctx.author, "-", 0x33
    await ctx.send(f"Cleared warnings for **{member}**.")


@bot.hybrid_command(description="Set warn thresholds (0 disables). e.g. 3 5")
@commands.guild_only()
@commands.has_permissions(administrator=True)
async def warnpunish(ctx, mute_at: int, ban_at: int):
    put(ctx.guild.id, "warn_mute", mute_at)
    put(ctx.guild.id, "warn_ban", ban_at)
    await ctx.send(f"Auto-mute at **{mute_at}** warns, auto-ban at **{ban_at}** (


@bot.hybrid_command(description="Delete the last N messages (max 100)")
```

```
@commands.guild_only()
@commands.has_permissions(manage_messages=True)
@commands.bot_has_permissions(manage_messages=True)
async def purge(ctx, amount: int):
    await ctx.defer(ephemeral=True)
    deleted = await ctx.channel.purge(limit=max(1, min(amount, 100)))
    await ctx.send(f"Deleted {len(deleted)} messages.", delete_after=5)


# ------------------------------------------------------------ utility
@bot.hybrid_command(description="Clear recent bot messages and bot commands")
@commands.guild_only()
@commands.has_permissions(manage_messages=True)
@commands.bot_has_permissions(manage_messages=True)
async def botclear(ctx):
    await ctx.defer(ephemeral=True)
    deleted = await ctx.channel.purge(
        limit=100, check=lambda m: m.author.bot or m.content.startswith(BOT_PREFI
    await ctx.send(f"🧹 Cleared {len(deleted)} bot messages.", delete_after=5)


@bot.hybrid_command(description="Translate text (default English). Reply to a mes
async def translate(ctx, to: str = "en", *, text: Optional[str] = None):
    if not text:
        ref = ctx.message.reference
        if ref and isinstance(ref.resolved, discord.Message):
            text = ref.resolved.content
    if not text:
        return await ctx.send(f"Usage: `{PREFIX}translate es hello there` or repl
    try:
        out = await asyncio.to_thread(GoogleTranslator(source="auto", target=to.l
    except Exception:
        return await ctx.send("Translation failed. Check the language code (en, e
    await ctx.send(embed=embed(f"Translation → {to.lower()}", out[:4000]))


@bot.hybrid_command(description="Check bot latency")
async def ping(ctx):
    await ctx.send(f"🐍 {round(bot.latency * 1000)}ms")


# tags
@bot.hybrid_group(invoke_without_command=True, description="Custom tags")
@commands.guild_only()
async def tag(ctx, *, name: Optional[str] = None):
    if not name:
```

```
        return await ctx.send(f"Usage: `{PREFIX}tag <name>`, `{PREFIX}tag create
                              f"`{PREFIX}tag delete <name>`, `{PREFIX}tag list`")
    await show_tag(ctx, name)


async def show_tag(ctx, name):
    r = db.execute("SELECT content FROM tags WHERE guild_id=? AND name=?", (ctx.g
    await ctx.send(r["content"] if r else "No such tag.", allowed_mentions=discor


@tag.command(name="show", description="Show a tag")
async def tag_show(ctx, *, name: str):
    await show_tag(ctx, name)


@tag.command(name="create", description="Create a tag")
async def tag_create(ctx, name: str, *, content: str):
    try:
        q("INSERT INTO tags VALUES(?,?,?,?)", ctx.guild.id, name.lower(), content
    except sqlite3.IntegrityError:
        return await ctx.send("A tag with that name already exists.")
    await ctx.send(f"Created tag `{name.lower()}`.")


@tag.command(name="delete", description="Delete a tag")
async def tag_delete(ctx, name: str):
    r = db.execute("SELECT author FROM tags WHERE guild_id=? AND name=?", (ctx.gu
    if not r:
        return await ctx.send("No such tag.")
    if r["author"] != ctx.author.id and not ctx.author.guild_permissions.manage_m
        return await ctx.send("Only the tag's author or a moderator can delete it
    q("DELETE FROM tags WHERE guild_id=? AND name=?", ctx.guild.id, name.lower())
    await ctx.send(f"Deleted tag `{name.lower()}`.")


@tag.command(name="list", description="List tags")
async def tag_list(ctx):
    rows = db.execute("SELECT name FROM tags WHERE guild_id=? ORDER BY name", (ct
    await ctx.send(", ".join(f"`{r['name']}`" for r in rows) or "No tags yet.")


# ---------------------------------------------------- setup / config
@bot.hybrid_command(description="Set a channel: modlog, serverlog, suggestions or
@commands.guild_only()
@commands.has_permissions(manage_guild=True)
async def setchannel(ctx, kind: Literal["modlog", "serverlog", "suggestions", "we
```

```
                    channel: discord.TextChannel):
    put(ctx.guild.id, kind, channel.id)
    await ctx.send(f"✅ **{kind}** → {channel.mention}")


@bot.hybrid_command(description="Set the welcome message. Use {user} {server} {co
@commands.guild_only()
@commands.has_permissions(manage_guild=True)
async def setwelcome(ctx, *, message: str):
    put(ctx.guild.id, "welcome_msg", message)
    await ctx.send("✅ Welcome message saved. Set the channel with `setchannel we


@bot.hybrid_command(description="Turn anti-raid or anti-nuke on/off")
@commands.guild_only()
@commands.has_permissions(administrator=True)
async def security(ctx, feature: Literal["antiraid", "antinuke"], state: Literal[
    put(ctx.guild.id, feature, state)
    await ctx.send(f"🛡️ **{feature}** is now **{state}**.")


@bot.hybrid_command(description="Server owner only: manage the anti-nuke whitelis
@commands.guild_only()
async def nukewhitelist(ctx, action: Literal["add", "remove", "list"], user: Opti
    if ctx.author.id != ctx.guild.owner_id:
        return await ctx.send("Only the server owner can do that.")
    if action == "list":
        rows = db.execute("SELECT user_id FROM whitelist WHERE guild_id=?", (ctx.
        return await ctx.send(", ".join(f"<@{r['user_id']}>" for r in rows) or "W
    if not user:
        return await ctx.send("Specify a user.")
    if action == "add":
        q("INSERT OR IGNORE INTO whitelist VALUES(?,?)", ctx.guild.id, user.id)
    else:
        q("DELETE FROM whitelist WHERE guild_id=? AND user_id=?", ctx.guild.id, u
    await ctx.send(f"Whitelist updated: {action} **{user}**.")


# ---------------------------------------------------------- giveaways
@bot.hybrid_command(description="Start a giveaway, e.g. /gstart 1h 1 Nitro")
@commands.guild_only()
@commands.has_permissions(manage_guild=True)
async def gstart(ctx, duration: str, winners: int, *, prize: str):
    secs = parse_dur(duration)
    if not secs or not 1 <= winners <= 20:
        return await ctx.send(f"Usage: `{PREFIX}gstart 1h 1 Nitro` (time units s/
```

```
    end = int(time.time() + secs)
    e = embed(f"🎉 {prize}", f"React with 🎉 to enter!\nEnds <t:{end}:R>\nWinners
                              f"Hosted by {ctx.author.mention}")
    msg = await ctx.channel.send(embed=e)
    await msg.add_reaction("🎉")
    q("INSERT INTO giveaways VALUES(?,?,?,?,?,?,0,?)",
      msg.id, ctx.guild.id, ctx.channel.id, prize, winners, end, ctx.author.id)
    if ctx.interaction:
        await ctx.send("Giveaway started!", ephemeral=True)


async def draw(row):
    ch = bot.get_channel(row["channel_id"])
    if not ch:
        return None, []
    try:
        msg = await ch.fetch_message(row["message_id"])
    except discord.NotFound:
        return None, []
    users = []
    for r in msg.reactions:
        if str(r.emoji) == "🎉":
            users = [u async for u in r.users() if not u.bot]
    return msg, random.sample(users, min(len(users), row["winners"]))


async def end_giveaway(row):
    q("UPDATE giveaways SET ended=1 WHERE message_id=?", row["message_id"])
    msg, winners = await draw(row)
    if not msg:
        return
    text = ", ".join(w.mention for w in winners) or "nobody (no valid entries)"
    e = embed(f"🎉 {row['prize']} (ended)", f"Winner(s): {text}\nHosted by <@{row
    await msg.edit(embed=e)
    await msg.channel.send(f"🎉 Giveaway over! Winner(s) of **{row['prize']}**: {


@bot.hybrid_command(description="End a giveaway now (message ID)")
@commands.guild_only()
@commands.has_permissions(manage_guild=True)
async def gend(ctx, message_id: str):
    row = db.execute("SELECT * FROM giveaways WHERE message_id=? AND guild_id=? A
                     (int(message_id), ctx.guild.id)).fetchone() if message_id.is
    if not row:
        return await ctx.send("No active giveaway with that message ID.")
    await end_giveaway(row)
```

```
    await ctx.send("Ended.", ephemeral=True)


@bot.hybrid_command(description="Reroll a finished giveaway (message ID)")
@commands.guild_only()
@commands.has_permissions(manage_guild=True)
async def greroll(ctx, message_id: str):
    row = db.execute("SELECT * FROM giveaways WHERE message_id=? AND guild_id=? A
                     (int(message_id), ctx.guild.id)).fetchone() if message_id.is
    if not row:
        return await ctx.send("No finished giveaway with that message ID.")
    _, winners = await draw(row)
    text = ", ".join(w.mention for w in winners) or "nobody"
    await ctx.send(f"🎉 New winner(s) of **{row['prize']}**: {text}")


@tasks.loop(seconds=15)
async def giveaway_loop():
    rows = db.execute("SELECT * FROM giveaways WHERE ended=0 AND end_ts<=?", (int
    for row in rows:
        try:
            await end_giveaway(row)
        except Exception:
            log.exception("giveaway end failed")


@giveaway_loop.before_loop
async def _wait_ready():
    await bot.wait_until_ready()


# ---------------------------------------------------------- suggestions
@bot.hybrid_command(description="Submit a suggestion")
@commands.guild_only()
async def suggest(ctx, *, suggestion: str):
    cid = get(ctx.guild.id, "suggestions")
    ch = ctx.guild.get_channel(int(cid)) if cid else discord.utils.get(ctx.guild.
    if not ch:
        return await ctx.send(f"No suggestions channel. An admin can run `{PREFIX
    sid = q("INSERT INTO suggestions(guild_id,message_id,channel_id,user_id,conte
            ctx.guild.id, ctx.author.id, suggestion).lastrowid
    e = embed(f"Suggestion #{sid}", suggestion[:4000])
    e.set_author(name=str(ctx.author), icon_url=ctx.author.display_avatar.url)
    msg = await ch.send(embed=e)
    for em in ("👍", "👎"):
        await msg.add_reaction(em)
```

```
    q("UPDATE suggestions SET message_id=?, channel_id=? WHERE id=?", msg.id, ch.
    await ctx.send(f"Suggestion #{sid} sent to {ch.mention}.", ephemeral=True)


async def resolve_suggestion(ctx, sid, approved, reason):
    r = db.execute("SELECT * FROM suggestions WHERE id=? AND guild_id=?", (sid, c
    ch = ctx.guild.get_channel(r["channel_id"]) if r else None
    if not ch:
        return await ctx.send("Suggestion not found.")
    try:
        msg = await ch.fetch_message(r["message_id"])
    except discord.NotFound:
        return await ctx.send("Couldn't find that suggestion's message.")
    e = msg.embeds[0]
    e.title = f"Suggestion #{sid} — {'Approved ✅' if approved else 'Denied ❌'}"
    e.color = 0x33AA55 if approved else 0xCC3333
    e.add_field(name=f"Reason (by {ctx.author})", value=reason[:1000], inline=Fal
    await msg.edit(embed=e)
    await ctx.send(f"Suggestion #{sid} {'approved' if approved else 'denied'}.",


@bot.hybrid_command(description="Approve a suggestion")
@commands.guild_only()
@commands.has_permissions(manage_guild=True)
async def approve(ctx, suggestion_id: int, *, reason: str = "No reason given"):
    await resolve_suggestion(ctx, suggestion_id, True, reason)


@bot.hybrid_command(description="Deny a suggestion")
@commands.guild_only()
@commands.has_permissions(manage_guild=True)
async def deny(ctx, suggestion_id: int, *, reason: str = "No reason given"):
    await resolve_suggestion(ctx, suggestion_id, False, reason)


# ---------------------------------------------------------- welcomer
@bot.listen("on_member_join")
async def welcomer(member):
    cid = get(member.guild.id, "welcome")
    ch = member.guild.get_channel(int(cid)) if cid else None
    if not ch:
        return
    text = get(member.guild.id, "welcome_msg", "Welcome {user} to **{server}**! Y
    text = (text.replace("{user}", member.mention).replace("{server}", member.gui
            .replace("{count}", str(member.guild.member_count)))
    try:
```

```
            await ch.send(text)
        except discord.HTTPException:
            pass


# ---------------------------------------------------------- anti-raid
joins = defaultdict(deque)
raid_until = {}


async def kick_raider(member, why):
    try:
        await member.kick(reason=f"Ouroboros anti-raid: {why}")
    except discord.HTTPException:
        pass


@bot.listen("on_member_join")
async def raid_guard(member):
    g = member.guild
    if get(g.id, "antiraid") != "on":
        return
    now = time.time()
    dq = joins[g.id]
    dq.append((now, member))
    while dq and now - dq[0][0] > RAID_WINDOW:
        dq.popleft()
    if now < raid_until.get(g.id, 0):
        return await kick_raider(member, "raid mode active")
    if len(dq) >= RAID_JOINS:
        raid_until[g.id] = now + RAID_LOCK
        victims = [m for _, m in dq]
        dq.clear()
        await send_log(g, "serverlog", embed(
            "🚨 Raid detected",
            f"{len(victims)} joins in {RAID_WINDOW}s. Kicking them; raid mode on
        for m in victims:
            await kick_raider(m, "join flood")


@bot.listen("on_message")
async def mention_spam(message):
    g = message.guild
    if not g or message.author.bot or get(g.id, "antiraid") != "on":
        return
    if len({u.id for u in message.mentions}) < MENTION_LIMIT:
```

```
        return
    if message.author.guild_permissions.manage_messages:
        return
    try:
        await message.delete()
        await message.author.timeout(timedelta(hours=1), reason="Ouroboros: mass
    except discord.HTTPException:
        pass
    await log_action(g, "Mass-mention timeout (1h)", message.author, bot.user, "A


# ------------------------------------------------------------ anti-nuke
nuke_hits = defaultdict(deque)
A = discord.AuditLogAction


async def audit_executor(guild, action, target_id):
    try:
        async for entry in guild.audit_logs(limit=6, action=action):
            age = (discord.utils.utcnow() - entry.created_at).total_seconds()
            if age < 15 and getattr(entry.target, "id", None) == target_id:
                return entry.user
    except discord.HTTPException:
        pass
    return None


async def nuke_check(guild, key, action, target_id):
    if get(guild.id, "antinuke") != "on":
        return
    await asyncio.sleep(1)  # give the audit log time to update
    user = await audit_executor(guild, action, target_id)
    if not user or user.id in (bot.user.id, guild.owner_id):
        return
    if db.execute("SELECT 1 FROM whitelist WHERE guild_id=? AND user_id=?", (guil
        return
    now = time.time()
    dq = nuke_hits[(guild.id, user.id, key)]
    dq.append(now)
    while dq and now - dq[0] > NUKE_WINDOW:
        dq.popleft()
    if len(dq) < NUKE_LIMITS[key]:
        return
    dq.clear()
    result = "could NOT be punished (check my role position)"
    try:
```

```
            await guild.ban(user, reason=f"Ouroboros anti-nuke: mass {key}")
            result = "was **banned**"
        except discord.HTTPException:
            member = guild.get_member(user.id)
            try:
                if member:
                    await member.edit(roles=[], reason=f"Ouroboros anti-nuke: mass {k
                    result = "had all roles **stripped**"
            except discord.HTTPException:
                pass
        await send_log(guild, "serverlog", embed(
            "🚨 Anti-nuke triggered", f"{user.mention} (`{user.id}`) did mass **{key}
        await send_log(guild, "modlog", embed(
            "🚨 Anti-nuke triggered", f"{user.mention} (`{user.id}`) did mass **{key}


@bot.listen("on_guild_channel_delete")
async def nuke_channel_delete(ch):
    await nuke_check(ch.guild, "channel_delete", A.channel_delete, ch.id)


@bot.listen("on_guild_channel_create")
async def nuke_channel_create(ch):
    await nuke_check(ch.guild, "channel_create", A.channel_create, ch.id)


@bot.listen("on_guild_role_delete")
async def nuke_role_delete(role):
    await nuke_check(role.guild, "role_delete", A.role_delete, role.id)


@bot.listen("on_guild_role_create")
async def nuke_role_create(role):
    await nuke_check(role.guild, "role_create", A.role_create, role.id)


@bot.listen("on_member_ban")
async def nuke_ban(guild, user):
    await nuke_check(guild, "ban", A.ban, user.id)


@bot.listen("on_member_remove")
async def nuke_kick(member):
    await nuke_check(member.guild, "kick", A.kick, member.id)
```

```
# ------------------------------------------------------------ server logs
@bot.listen("on_message_delete")
async def log_delete(m):
    if not m.guild or m.author.bot:
        return
    await send_log(m.guild, "serverlog", embed(
        "Message deleted",
        f"**Author:** {m.author.mention}\n**Channel:** {m.channel.mention}\n\n{(m
        0xCC4444))


@bot.listen("on_message_edit")
async def log_edit(before, after):
    if not after.guild or after.author.bot or before.content == after.content:
        return
    await send_log(after.guild, "serverlog", embed(
        "Message edited",
        f"**Author:** {after.author.mention}\n**Channel:** {after.channel.mention
        f"**Before:** {before.content[:700]}\n**After:** {after.content[:700]}",


@bot.listen("on_member_join")
async def log_join(m):
    await send_log(m.guild, "serverlog", embed(
        "Member joined", f"{m.mention} (`{m.id}`)\nAccount created <t:{int(m.crea


@bot.listen("on_member_remove")
async def log_leave(m):
    await send_log(m.guild, "serverlog", embed("Member left", f"{m} (`{m.id}`)",


@bot.listen("on_member_update")
async def log_member_update(before, after):
    if before.roles != after.roles:
        added = [r.mention for r in after.roles if r not in before.roles]
        removed = [r.mention for r in before.roles if r not in after.roles]
        desc = f"{after.mention}\n" + (f"**Added:** {', '.join(added)}\n" if adde
               (f"**Removed:** {', '.join(removed)}" if removed else "")
        await send_log(after.guild, "serverlog", embed("Roles updated", desc))
    if before.nick != after.nick:
        await send_log(after.guild, "serverlog", embed(
            "Nickname changed", f"{after.mention}: `{before.nick}` → `{after.nick


@bot.listen("on_guild_channel_create")
```

```
async def log_ch_create(ch):
    await send_log(ch.guild, "serverlog", embed("Channel created", f"{ch.mention}


@bot.listen("on_guild_channel_delete")
async def log_ch_delete(ch):
    await send_log(ch.guild, "serverlog", embed("Channel deleted", f"`#{ch.name}`


@bot.listen("on_guild_role_create")
async def log_role_create(r):
    await send_log(r.guild, "serverlog", embed("Role created", f"{r.mention} (`{r


@bot.listen("on_guild_role_delete")
async def log_role_delete(r):
    await send_log(r.guild, "serverlog", embed("Role deleted", f"`{r.name}`", 0xC


@bot.listen("on_member_ban")
async def log_ban(guild, user):
    await send_log(guild, "serverlog", embed("Member banned", f"{user} (`{user.id


@bot.listen("on_member_unban")
async def log_unban(guild, user):
    await send_log(guild, "serverlog", embed("Member unbanned", f"{user} (`{user.


@bot.listen("on_voice_state_update")
async def log_voice(m, before, after):
    if before.channel == after.channel:
        return
    if not before.channel:
        desc = f"{m.mention} joined {after.channel.mention}"
    elif not after.channel:
        desc = f"{m.mention} left {before.channel.mention}"
    else:
        desc = f"{m.mention} moved {before.channel.mention} → {after.channel.ment
    await send_log(m.guild, "serverlog", embed("Voice", desc))


# ------------------------------------------------------------ errors
@bot.event
async def on_command_error(ctx, error):
    error = getattr(error, "original", error)
```

```
    if isinstance(error, commands.CommandNotFound):
        return
    if isinstance(error, commands.MissingPermissions):
        msg = "You don't have permission to do that."
    elif isinstance(error, commands.BotMissingPermissions):
        msg = "I'm missing permissions: " + ", ".join(error.missing_permissions)
    elif isinstance(error, commands.MissingRequiredArgument):
        msg = f"Missing argument: `{error.param.name}`. Try `{PREFIX}help {ctx.co
    elif isinstance(error, (commands.BadArgument, commands.BadLiteralArgument)):
        msg = "I couldn't understand one of those arguments."
    elif isinstance(error, commands.NoPrivateMessage):
        msg = "That only works in a server."
    elif isinstance(error, commands.CheckFailure):
        msg = "You can't use that."
    else:
        log.exception("Command error", exc_info=error)
        msg = "Something went wrong."
    try:
        await ctx.send(msg, ephemeral=True)
    except discord.HTTPException:
        pass


@bot.event
async def on_ready():
    log.info("Ouroboros online as %s in %d servers", bot.user, len(bot.guilds))


bot.run(TOKEN, log_handler=None)
```